

لنظام التسجيل (Threat Modeling) تمرين تطبيقي والشراء

1. تحديد الأصول (Assets)

أ) البيانات:

- معلومات شخصية، بريد إلكتروني، كلمة مرور: بيانات المستخدمين
- أرقام بطاقات الائتمان، معلومات الدفع: بيانات الدفع
- الأسعار، المخزون، التفاصيل: بيانات المنتجات
- تاريخ الشراء، التفاصيل، المبالغ: سجلات المعاملات
- سجلات المراجعة Logs: سجلات النظام

ب) النظم:

- PHP مع تطبيق Apache/Nginx: خادم الويب
- MySQL/PostgreSQL: قاعدة البيانات
- بوابة الدفع: الدفع الإلكتروني
- نظام المصادقة والتفويض: إدارة الجلسات

ج) الوظائف:

- التسجيل، التسوق، الشراء: وظائف المستخدم العادي
- إدارة المستخدمين، المنتجات، التقارير: وظائف المدير

2. نقاط الدخول Entry Points تحديد

أ) الواجهات الأمامية:

text

1. POST /register صفحة التسجيل
2. POST /login صفحة تسجيل الدخول
3. POST /reset-password استعادة كلمة المرور
4. POST /cart/add إضافة منتج للسلة

5. POST /checkout عملية الشراء
6. POST /payment/process الدفع
7. GET/POST /admin/* لوحة التحكم
8. واجهات API /api/v1/*

ب) نماذج الإدخال

php

// نموذج التسجيل

```
<form action="/register" method="POST">
    <input name="username">
    <input name="email">
    <input name="password">
    <input name="confirm_password">
</form>
```

// نموذج الدفع

```
<form action="/payment" method="POST">
    <input name="card_number">
    <input name="expiry">
    <input name="cvv">
</form>
```

3. STRIDE تطبيق

STRIDE جدول تحليل

التهديد	الوصف	الأمثلة في النظام
Spoofting	انتحال الهوية	تسجيل دخول غير مصرح - انتحال جلسة - CSRF انتحال طلب -

التهديد	الوصف	الأمثلة في النظام
Tampering	التلاعب بالبيانات	- تعديل الأسعار - تغيير صلاحيات المستخدم - API تلاعب في طلبات
Repudiation	الإنكار	- إنكار عملية شراء - إنكار تعديل منتج - إنكار حذف مستخدم
Information Disclosure	كشف المعلومات	- كشف بيانات الدفع - تسريب بيانات المستخدمين - عرض أخطاء مفصلة
Denial of Service	حجب الخدمة	- DDoS هجوم - إغراق قاعدة البيانات - إبطاء النظام
Elevation of Privilege	تصعيد الصلاحيات	- وصول مستخدم عادي لوظائف المدير - تجاوز التحقق من الصلاحيات

تحليل تفصيلي لكل نقطة دخول

1. نقطة الدخول: التسجيل (POST /register)

text

تهديدات STRIDE:

- S: إنشاء حسابات وهمية (Bots)
- T: حقن بيانات خاطئة في قاعدة البيانات
- I: كشف إذا كان البريد مسبقاً
- D: إغراق النظام بطلبات تسجيل

2. نقطة الدخول: تسجيل الدخول (POST /login)

text

تهديدات STRIDE:

- S: محاولات تخمين كلمات المرور

- T: في استعلام الدخول SQL حقن
- R: إنكار محاولة تسجيل دخول فاشلة
- I: كشف معلومات النظام عبر رسائل الخطأ
- D: لتجنب الحساب Brute Force هجوم

3. نقطة الدخول: الدفع (POST /payment)

text

تهديدات STRIDE:

- S: انتحال طلب دفع
- T: تعديل قيمة المبلغ أو المنتجات
- R: إنكار عملية الشراء
- I: كشف بيانات الدفع الحساسة
- E: تجاوز التحقق من الدفع

4. نقطة الدخول: لوحة المدير (/admin/*)

text

تهديدات STRIDE:

- S: انتحال هوية المدير
- T: تعديل بيانات المستخدمين/المنتجات
- R: إنكار التعديلات
- I: كشف معلومات حساسة للمستخدمين
- E: وصول مستخدم عادي للوحة التحكم

4. اقتراح ضوابط أمنية

(أ) ضوابط عامة للنظام

php

```
// إعدادات أمنية أساسية 1.
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
ini_set('session.use_strict_mode', 1);
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: DENY');
header('X-XSS-Protection: 1; mode=block');
```

ب) ضوابط لكل نقطة دخول

نقطة الدخول 1: التسجيل

php

```
class RegistrationSecurity {
    public function validateRegistration($data) {
        // 1. CAPTCHA لمنع Bots
        if (!$this->verifyCaptcha($data['captcha'])) {
            throw new Exception('الرجاء إثبات أنك لست روبوت');
        }

        // 2. Rate Limiting
        $this->rateLimit('registration', $_SERVER['REMOTE_ADDR']);

        // 3. تحقق من صحة البيانات
        if (!$this->isValidEmail($data['email'])) {
            throw new Exception('البريد الإلكتروني غير صالح');
        }

        if (strlen($data['password']) < 8) {
            throw new Exception('كلمة المرور يجب أن تكون 8 أحرف على الأقل');
        }

        // 4. منع إنشاء حسابات متعددة لنفس الشخص
        if ($this->hasRecentRegistration($_SERVER['REMOTE_ADDR'])) {
            throw new Exception('لقد أنشأت حساباً مؤخراً');
        }

        // 5. تشفير كلمة المرور
        $hashed_password = password_hash($data['password'], PASSWORD_BCRYPT);

        // 6. تأكيد البريد الإلكتروني
        $this->sendVerificationEmail($data['email']);

        return true;
    }

    private function rateLimit($action, $identifier) {
```

```

        // تطبيق Rate Limiting
    }
}

```

نقطة الدخول 2: تسجيل الدخول

php

```

class LoginSecurity {
    public function secureLogin($username, $password) {
        // 1. Rate Limiting أكثر صرامة
        if ($this->isRateLimited($username)) {
            throw new Exception('محاولات دخول كثيرة. حاول بعد 15 دقيقة');
        }

        // 2. Prepared Statement لمنع SQL Injection
        $stmt = $this->db->prepare("
            SELECT id, password, failed_attempts, locked_until
            FROM users
            WHERE username = ? OR email = ?
        ");
        $stmt->execute([$username, $username]);
        $user = $stmt->fetch();

        // 3. التحقق من الحساب المقفل
        if ($user['locked_until'] && time() < strtotime($user['locked_until']))
        ) {
            throw new Exception('الحساب مؤقتاً مغلق');
        }

        // 4. التحقق من كلمة المرور
        if (!password_verify($password, $user['password'])) {
            // زيادة عداد المحاولات الفاشلة
            $this->incrementFailedAttempts($user['id']);

            // قفل الحساب بعد 5 محاولات فاشلة
            if ($user['failed_attempts'] + 1 >= 5) {
                $this->lockAccount($user['id'], 15); // قفل لمدة 15 دقيقة
            }
        }
    }
}

```

```

        throw new Exception('بيانات الدخول غير صحيحة');
    }

    // 5. إعادة تعيين المحاولات الفاشلة
    $this->resetFailedAttempts($user['id']);

    // 6. تسجيل جلسة آمنة
    $this->createSecureSession($user['id']);

    // 7. تسجيل محاولة الدخول الناجحة
    $this->logLoginAttempt($user['id'], true);

    return true;
}
}

```

نقطة الدخول 3: الدفع

php

```

class PaymentSecurity {
    public function processPayment($order_id, $payment_data) {
        // 1. التحقق من CSRF Token
        if (!$this->validateCSRFToken()) {
            throw new Exception('طلب غير صالح');
        }

        // 2. التحقق من ملكية الطلب
        if (!$this->verifyOrderOwnership($order_id, $_SESSION['user_id'])) {
            throw new Exception('غير مصرح بالوصول لهذا الطلب');
        }

        // 3. تقييد معالجة نفس الطلب مرتين
        if ($this->isOrderAlreadyProcessed($order_id)) {
            throw new Exception('تم معالجة هذا الطلب مسبقاً');
        }

        // 4. Rate Limiting لعملية الدفع
        $this->rateLimitPayment($_SESSION['user_id']);
    }
}

```

```

// 5. تظهير بيانات الدفع (Tokenization)
$token = $this->tokenizePaymentData($payment_data);

// 6. إرسال للبوابة الآمنة فقط
$response = $this->sendToPaymentGateway($token);

// 7. فريد ID تسجيل المعاملة مع
$transaction_id = $this->logTransaction([
    'order_id' => $order_id,
    'user_id' => $_SESSION['user_id'],
    'amount' => $this->getOrderAmount($order_id),
    'status' => $response['status'],
    'reference' => $response['reference']
]);

// 8. إخطار المستخدم
$this->sendPaymentConfirmation($_SESSION['user_id'], $transaction_id);

return $transaction_id;
}
}

```

نقطة الدخول 4: لوحة التحكم

php

```

class AdminSecurity {
    public function accessControl($required_role) {
        // التحقق من تسجيل الدخول
        if (!isset($_SESSION['user_id'])) {
            header('Location: /login');
            exit;
        }

        // التحقق من الصلاحيات
        if (!$this->hasRole($_SESSION['user_id'], $required_role)) {
            throw new Exception('غير مصرح بالوصول');
        }

        // المسموح به (اختياري للمديرين) IP التحقق من
    }
}

```



```

        if (!$this->isAllowedIP($_SERVER['REMOTE_ADDR'])) {
            $this->logSuspiciousAccess($_SESSION['user_id']);
            throw new Exception('الوصول من هذا العنوان غير مسموح');
        }

        // 4. تسجيل نشاط المدير
        $this->logAdminActivity($_SESSION['user_id'], $_SERVER['REQUEST_URI'])
    ;

    // إعادة المصادقة للعمليات الحساسة 5.
    if ($this->isSensitiveOperation($_SERVER['REQUEST_URI'])) {
        if (!$this->reauthenticate()) {
            throw new Exception('المصادقة المطلوبة');
        }
    }

    return true;
}

private function hasRole($user_id, $required_role) {
    // التحقق من الصلاحيات في قاعدة البيانات
    $stmt = $this->db->prepare("
        SELECT role
        FROM users
        WHERE id = ?
        AND role = ?
    ");
    $stmt->execute([$user_id, $required_role]);

    return $stmt->rowCount() > 0;
}
}
}

```

STRIDE: ضوابط حسب تهديدات

Spoofing: الحماية من

php

```
class AntiSpoofing {
```

```

public function preventSessionHijacking() {
    // ومتصفح المستخدم IP ربط الجلسة بعنوان
    session_start();

    $session_fingerprint = hash('sha256',
        $_SERVER['HTTP_USER_AGENT'] .
        $_SERVER['REMOTE_ADDR']
    );

    if (!isset($_SESSION['fingerprint'])) {
        $_SESSION['fingerprint'] = $session_fingerprint;
    } elseif ($_SESSION['fingerprint'] !== $session_fingerprint) {
        // جلسة مشبوهة
        session_regenerate_id(true);
        session_destroy();
        header('Location: /login?error=hijack');
        exit;
    }
}

public function implement2FA($user_id) {
    // توثيق متعدد العوامل
    $secret = $this->get2FAsecret($user_id);

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $user_code = $_POST['2fa_code'];

        if ($this->verify2FAcode($secret, $user_code)) {
            $_SESSION['2fa_verified'] = true;
            return true;
        }

        throw new Exception('رمز التحقق غير صحيح');
    }

    return false;
}
}

```



```

        throw new Exception("يجب أن يكون رقماً الحقل $field");
    }
    break;

    case 'email':
        if (!filter_var($input[$field], FILTER_VALIDATE_EMAIL)) {
            throw new Exception("بريد إلكتروني غير صالح");
        }
        break;

    case 'alphanumeric':
        if (!preg_match('/^[a-zA-Z0-9]+$', $input[$field])) {
            throw new Exception("يجب أن يحتوي على أ الحقل $field  
("حرف وأرقام فقط");
        }
        break;
    }
}

return true;
}
}

```

Repudiation: الحماية من

php

```

class NonRepudiation {
    public function createAuditLog($action, $user_id, $details) {
        // سجلات تدقيق لا يمكن التلاعب بها
        $log_entry = [
            'timestamp' => time(),
            'action' => $action,
            'user_id' => $user_id,
            'ip_address' => $_SERVER['REMOTE_ADDR'],
            'user_agent' => $_SERVER['HTTP_USER_AGENT'],
            'details' => $details
        ];
    }
}

```

```

        // تشفير السجلات
        $encrypted_log = $this->encryptLog($log_entry);

        // تخزين في قاعدة بيانات آمنة
        $this->storeLog($encrypted_log);

        // إضافة إلى سلسلة بلوكشين (للسجلات شديدة الأهمية)
        $this->addToBlockchain($log_entry);
    }

    public function signTransaction($user_id, $transaction_data) {
        // توقيع رقمي للمعاملات
        $private_key = $this->getUserPrivateKey($user_id);

        $data_to_sign = json_encode([
            'user_id' => $user_id,
            'data' => $transaction_data,
            'timestamp' => time()
        ]);

        openssl_sign($data_to_sign, $signature, $private_key, OPENSSL_ALGO_SHA
256);

        return [
            'data' => $transaction_data,
            'signature' => base64_encode($signature),
            'timestamp' => time()
        ];
    }
}

```

Information Disclosure: الحماية من

php

```

class DataProtection {
    public function encryptSensitiveData($data, $user_id) {
        // تشفير البيانات الحساسة
        $key = $this->deriveUserKey($user_id);
        $iv = random_bytes(16);
    }
}

```

```

        $encrypted = openssl_encrypt(
            $data,
            'aes-256-gcm',
            $key,
            OPENSSSL_RAW_DATA,
            $iv,
            $tag
        );

        return base64_encode($iv . $tag . $encrypted);
    }

    public function maskCreditCard($card_number) {
        // إخفاء أرقام البطاقة
        return '**** *' . substr($card_number, -4);
    }

    public function safeErrorHandling($exception) {
        // معالجة أخطاء آمنة
        error_log('ERROR: ' . $exception->getMessage() .
            ' in ' . $exception->getFile() .
            ' on line ' . $exception->getLine());

        // رسالة آمنة للمستخدم
        if (ENVIRONMENT === 'production') {
            return 'حدث خطأ في النظام. الرجاء المحاولة لاحقاً';
        }

        return $exception->getMessage();
    }
}

```

Denial of Service: الحماية من

php

```

class DoSProtection {
    public function applyRateLimiting() {
        // متعدد المستويات Rate Limiting
    }
}

```

```

$ip = $_SERVER['REMOTE_ADDR'];
$user_id = $_SESSION['user_id'] ?? 'anonymous';

// 1. Rate Limiting على مستوى IP
if (!$this->checkRateLimit('ip', $ip, 100, 60)) {
    throw new Exception('Too many requests from your IP');
}

// 2. Rate Limiting على مستوى المستخدم
if ($user_id !== 'anonymous') {
    if (!$this->checkRateLimit('user', $user_id, 50, 60)) {
        throw new Exception('Too many requests from your account');
    }
}

// 3. Rate Limiting على نقطة النهاية
$endpoint = $_SERVER['REQUEST_URI'];
if (!$this->checkRateLimit('endpoint', $endpoint, 20, 10)) {
    throw new Exception('Too many requests to this endpoint');
}

return true;
}

public function validateRequestSize() {
    // التحقق من حجم الطلب
    $max_size = 10 * 1024 * 1024; // 10MB

    if ($_SERVER['CONTENT_LENGTH'] > $max_size) {
        throw new Exception('Request too large');
    }

    // التحقق من عدد المعاملات
    if (count($_POST) > 100) {
        throw new Exception('Too many parameters');
    }
}
}
}

```

Elevation of Privilege: الحماية من

php

```
class PrivilegeControl {
    public function enforceLeastPrivilege($user_id, $action) {
        // تطبيق مبدأ أقل صلاحية
        $required_permission = $this->getPermissionForAction($action);
        $user_permissions = $this->getUserPermissions($user_id);

        if (!$this->hasPermission($user_permissions, $required_permission)) {
            throw new Exception('Insufficient permissions');
        }

        // التحقق من السياق
        if ($action === 'edit_user' && !$this->canEditUser($user_id, $_GET['target_user'])) {
            throw new Exception('Cannot edit this user');
        }

        return true;
    }

    public function implementRoleBasedAccessControl() {
        // RBAC
        $roles = [
            'admin' => [
                'view_users', 'edit_users', 'delete_users',
                'view_products', 'edit_products', 'delete_products'
            ],
            'moderator' => [
                'view_users', 'edit_users',
                'view_products', 'edit_products'
            ],
            'customer' => [
                'view_products', 'purchase', 'view_orders'
            ]
        ];

        $user_role = $_SESSION['role'] ?? 'customer';
```



```

        $required_permission = $this->getCurrentAction();

        if (!in_array($required_permission, $roles[$user_role])) {
            header('HTTP/1.0 403 Forbidden');
            exit;
        }
    }
}
}

```

خطة التنفيذ المرحلي:

المرحلة 1: الأساسيات (الضرورية)

1. ✓ Prepared Statements لجميع الاستعلامات
2. ✓ password_hash التشفير بكلمة المرور
3. ✓ CSRF Tokens للنماذج
4. ✓ Rate Limiting للتسجيل والدخول

المرحلة 2: المستوى المتوسط

1. ✓ توقيع الجلسات
2. ✓ سجلات التدقيق
3. ✓ تحقق من ملكية الموارد
4. ✓ XSS حماية من

المرحلة 3: المستوى المتقدم

1. ✓ توثيق متعدد العوامل للمديرين
2. ✓ تشفير البيانات الحساسة
3. ✓ WAF (Web Application Firewall)
4. ✓ مراقبة الأحداث الأمنية

خطة المراقبة والاستجابة:

```

php
class SecurityMonitoring {

```

```

public function monitorForThreats() {
    $suspicious_patterns = [
        '/union.*select/i' => 'SQL Injection attempt',
        '/<script.*>/i' => 'XSS attempt',
        '/\\.\\.\\.\\/\\/' => 'Path traversal attempt',
        '/eval\\(/i' => 'Code injection attempt'
    ];

    foreach ($_REQUEST as $key => $value) {
        foreach ($suspicious_patterns as $pattern => $threat) {
            if (preg_match($pattern, $value)) {
                $this->alertSecurityTeam($threat, $key, $value);
                $this->blockIPIfNecessary($_SERVER['REMOTE_ADDR']);
            }
        }
    }
}

public function alertSecurityTeam($threat, $details) {
    // إرسال تنبيه لفريق الأمن
    $message = "THREAT DETECTED: $threat\n" .
        "IP: {$_SERVER['REMOTE_ADDR']}\n" .
        "URL: {$_SERVER['REQUEST_URI']}\n" .
        "Details: " . print_r($details, true);

    error_log($message);

    // Slack يمكن إرسال إيميل أو رسالة
    if ($this->isCriticalThreat($threat)) {
        $this->sendAlertToPagerDuty($message);
    }
}
}

```